Switching, Routing y Monitorización con Zabbix

Temario explicado · Módulos 1 a 6

Índice

1. Módulo 1 · Conceptos generales
2. Módulo 2 · IPv4 e IPv6
3. Módulo 3 · Cómo funciona la red
4. Módulo 4 · Switching
5. Módulo 5 · Routing
6. Módulo 6 · Zabbix

Módulo 1 · Conceptos generales

Objetivo del módulo: hablar el idioma básico de las redes sin sufrir. Al final sabréis clasificar una red, entender qué es eso de las "capas", reconocer los aparatos que hay en cualquier sala de comunicaciones y mirar la configuración de red de vuestro propio equipo.

1.1 Tipos de redes: cuatro formas de clasificarlas

Una red no es "una cosa": depende de cómo la mires. Igual que una persona puede ser "alta", "del Betis" y "de Jerez" a la vez, una red puede clasificarse de cuatro maneras distintas.

Según...	Ejemplos
Prestación del servicio	Líneas dedicadas, RTC, Dial-up, vía satélite
Topología (la forma)	Bus, anillo, estrella, punto a punto, hub-and-spoke, malla completa
Geografía (el tamaño)	LAN, MAN, WAN

Según...	Ejemplos
Recursos (quién manda)	Cliente-Servidor, P2P

Para no liarse:

- **LAN** (Local Area Network): vuestra casa, vuestra oficina. **MAN**: una ciudad. **WAN**: media humanidad (Internet es la WAN de las WAN).
- **Cliente-Servidor:** uno sirve y los demás piden (como un bar: el camarero sirve, vosotros pedís). **P2P:** todos son iguales y se comparten cosas entre sí (como una cena de tupper en la que cada uno trae algo).
- **Topologías:** la **estrella** (todos conectados a un punto central) es la reina de hoy. El **bus** (todos colgados del mismo cable) es una reliquia, como el walkman. **Malla completa**: todos con todos; muy robusta y muy cara (con 10 equipos necesitas 45 cables; haz la cuenta de 100).

1.2 Las 7 capas del modelo OSI

¿Para qué sirve un "modelo"?

Imaginad que enviar un correo fuera una sola tarea gigante. Imposible de arreglar cuando falla. Por eso se divide en **capas**: cada capa hace *una* cosa y se despreocupa del resto. Si falla algo, sabes *en qué piso* buscar.

Es como pedir una pizza: tú no sabes cómo se amasa, ni cómo conduce el repartidor, ni de qué está hecho su casco. Cada uno hace lo suyo y la pizza llega.

Nº	Capa	Qué hace (en cristiano)	Ejemplo
7	**Aplicación**	Lo que ve el usuario	Web, correo
6	**Presentación**	Formato, cifrado, compresión	Que el texto "ñ" no se convierta en "Ã±"
5	**Sesión**	Abre y cierra la conversación	"Hola, ¿me oyes?" … "Adiós"
4	**Transporte**	Entrega fiable (o rápida)	TCP / UDP
3	**Red**	Direcciones IP y mejor ruta	El "GPS"

Nº	Capa	Qué hace (en cristiano)	Ejemplo
2	**Enlace**	Direcciones MAC, dentro de la LAN	El "reparto por el barrio"
1	**Física**	Cables, señales, bits	Un cable, una fibra, una onda wifi

Truco mnemotécnico (de 7 a 1): *"**A** **P**residente **S**e le **T**rata de **R**ey **F**ísicamente"* → **A**plicación, **P**resentación, **S**esión, **T**ransporte, **R**ed, **E**nlace... (vale, aquí la regla falla un poquito con la E, pero el chiste se queda).

1.3 Curiosidad: ¿y si hubiera más capas?

Esto no es broma del todo. Hay una propuesta (de Ramana Kompella y Vijoy Pandey) para ampliar el modelo con dos capas más pensando en los **agentes de IA**:

Capa	Nombre	Para qué
8	**Comunicación de Agentes**	El *"cómo"* hablan los agentes de IA entre sí (por ejemplo MCP)
9	**Semántica de Agentes**	El *"qué"*: que no se malinterpreten entre ellos

La idea: igual que antes teníamos que poner de acuerdo a los ordenadores para que se entendieran, ahora hay que poner de acuerdo a las inteligencias artificiales.

(No confundir con las "capas 8 y 9" de la guasa informática: la capa 8 es "el usuario" y la capa 9 es "el jefe del usuario". Y esto último sí que es broma.)

1.4 Dos mapas, un mismo territorio: OSI vs. TCP/IP

OSI (7 capas)	TCP/IP (4 capas)
7 Aplicación	**Aplicación** (agrupa 5-7)
6 Presentación	
5 Sesión	

OSI (7 capas)	TCP/IP (4 capas)
4 Transporte	**Transporte**
3 Red	**Internet** (≈ Red)
2 Enlace	**Acceso a la red** (≈ 1-2)
1 Física	

- **OSI** es el **mapa de referencia**: el que se estudia para entender y para dar nombre a las cosas.
- **TCP/IP** es el que **de verdad se usa** en Internet.

Es como la diferencia entre el libro de recetas de cocina profesional y lo que realmente haces un martes a las diez de la noche: ambos sirven, pero uno es teoría y el otro es supervivencia.

1.5 Cómo viaja un mensaje: sobres dentro de sobres

Cuando envías un correo, los datos no viajan "desnudos". Cada capa los mete en un **sobre** y le pega su etiqueta. Esto se llama **encapsulación**.

```
Datos  →  Segmento  →  Paquete  →  Trama
          (+puertos)    (+IP)       (+MAC)
          Transporte    Red         Enlace
```

Nombre	Lo que añade	Capa
Datos	El mensaje en sí	Aplicación
Segmento	Puertos (¿a qué programa va?)	Transporte
Paquete	Direcciones IP (¿a qué máquina va?)	Red
Trama	Direcciones MAC (¿al siguiente salto?)	Enlace

Es como mandar una carta a un amigo: metes la carta en un sobre (segmento), el sobre en una caja con la dirección de la ciudad (paquete), y la caja en la furgoneta de reparto del barrio (trama). Cuando llega, el receptor va abriendo en orden inverso, como una muñeca rusa.

1.6 Parada y pensamos: ¿qué leches falla?

Situación: *"No tengo Internet."* Tres palabras que han arruinado millones de tardes.

La clave para diagnosticar: **repartir las causas entre capas bajas y capas altas.**

Capas bajas (física, enlace, red)	Capas altas (transporte, aplicación)
Cable desenchufado o roto	El navegador está colgado
Wifi desconectado	El DNS no resuelve nombres
IP mal configurada	El servidor web está caído
Puerta de enlace caída	El firewall bloquea ese puerto

Método rápido: empezad por abajo y subid. ¿Hay luz en el puerto? ¿Tengo IP? ¿Llego a mi router? ¿Llego a Internet por IP (por ejemplo `ping 8.8.8.8`)? ¿Resuelvo nombres (`ping google.com`)?

1.7 Dispositivos de infraestructura de red

Dispositivo	Qué hace	Analogía
Hub	Repite la señal a **todos** los puertos, sin filtrar	El que grita la noticia en la plaza del pueblo
Bridge	Conecta dos segmentos de LAN	Un puente entre dos barrios
Switch	Envía solo al puerto correcto, por MAC	El cartero que sabe en qué piso vives
Router	Conecta redes distintas, busca la mejor ruta	El GPS entre ciudades
Access Point	Crea la señal Wi-Fi para la red cableada	La "antena" de la red
Firewall	Bloquea tráfico peligroso	El portero de la discoteca
Módem	Traduce la señal digital para viajar por la línea	El intérprete entre dos idiomas

Dispositivo	Qué hace	Analogía
Repetidor	Amplifica la señal a mayor distancia	El que repite lo que dijiste porque estás lejos

---

1.8 Interfaces LAN: por dónde se conecta cada cosa

Tipo	Ejemplos
Cableadas	Ethernet RJ-45 (1-10 Gbps), Fibra SFP/SFP+ (10-100 Gbps), Coaxial BNC *(obsoleta)*
Inalámbricas	Wi-Fi (802.11, 2.4 / 5 / 6 GHz), Bluetooth (WPAN, corto alcance)
Lógicas / virtuales	VLAN, Subinterfaces (router-on-a-stick), Loopback
Otras	Serie (WAN antigua), Consola (gestión local)

Las **lógicas** son interfaces que no existen físicamente: son "enchufes imaginarios" que el equipo crea por software. Suena a magia, pero lo veréis clarísimo en los módulos de VLAN y routing.

> 😄 **El cable de consola:** es ese cable azul (o gris, o amarillo, según el fabricante) que todo técnico lleva en la mochila y que nadie sabe dónde está cuando lo necesita. Existe una ley no escrita: *el cable de consola está siempre en el último sitio donde miras*.

1.9 Practica ya: ¿qué veis en vuestra propia configuración de red?

Vamos a mirar nuestro propio equipo. Son cinco datos y los vais a usar todo el curso:

Dato	Dónde se mira
Dirección IP	`ipconfig /all` · `ip a`
Máscara de subred	`ipconfig /all` · `ip a`
Puerta de enlace	`ipconfig /all` · `ip route`

Dato	Dónde se mira
DNS	`ipconfig /all` · `resolv.conf`
Dirección MAC	`ipconfig /all` · `ip link`

- **Windows:** `ipconfig /all`
- **Linux / Mac:** `ip a` y `ip route`

Mini-glosario del módulo:

- **LAN/WAN:** red local / red extensa.
- **Encapsulación:** meter datos en "sobres" por capas.
- **Topología:** la forma de la red.
- **Puerta de enlace (gateway):** la salida de tu red hacia el exterior.

Módulo 2 · IPv4 e IPv6

Objetivo del módulo: entender las direcciones IP, calcular redes y subredes sin llorar, y saber por qué existe el NAT.

2.1 ¿Qué es una dirección IP?

Una dirección IP es el **"domicilio" de un equipo en la red**. Tiene dos partes:

```
192.168.1.10
└─ red ─┘ └host┘
```

- **Parte de red:** el "barrio" (a qué red pertenezco).
- **Parte de host:** el "número del portal" (quién soy dentro de esa red).

La **máscara de subred** es la que nos dice dónde se corta: qué parte es el barrio y cuál es el portal.

2.2 Clases de direcciones y máscara

Históricamente, las direcciones IPv4 se repartían en "clases":

Clase	Rango del primer número	Máscara por defecto
A	1 – 126	`255.0.0.0` (/8)
B	128 – 191	`255.255.0.0` (/16)
C	192 – 223	`255.255.255.0` (/24)

- El **127** está reservado para *loopback* (`127.0.0.1`, "yo mismo"; el famoso "*en mi casa funciona*").
- Hoy se usa el direccionamiento **sin clases (CIDR)**, que permite cortar donde nos dé la gana (ahí entra el subnetting), pero conviene conocer las clases porque seguimos oyendo hablar de ellas.

¿Qué es "/24"? Es el número de bits de la máscara dedicados a "red". `/24` = 24 bits de red y 8 bits de host.

2.3 Red, broadcast y hosts

En cada subred hay **tres tipos de direcciones**:

Dirección	Cómo se reconoce
Dirección de red	Todos los bits de host a **0**
Hosts útiles	Entre la red+1 y el broadcast-1
Dirección de broadcast	Todos los bits de host a **1**

Fórmula de oro: hosts útiles = $2^n - 2$ (donde *n* son los bits de host). El "−2" son la dirección de red y la de broadcast, que no se pueden asignar a ningún equipo.

Ejemplo resuelto: 192.168.5.0/24

- Bits de host: 8 → $2^8 - 2$ = **254 hosts útiles**
- Red: `192.168.5.0`
- Primer host: `192.168.5.1`
- Último host: `192.168.5.254`
- Broadcast: `192.168.5.255`

2.4 Subnetting y VLSM

Subnetting = trocear una red grande en redes más pequeñas.
VLSM (*Variable Length Subnet Mask*) = dar a **cada subred el tamaño justo que necesita**, ni más ni menos.

Es como repartir una tarta: no cortas porciones iguales para todos si uno tiene mucha hambre (Ventas) y otro solo quiere probar un trocito (Invitados).

Departamento	Hosts necesarios	Máscara	Hosts útiles que ofrece
Ventas	50	/26	62
Administración	20	/27	30
IT	10	/28	14
Invitados	5	/29	6

Cómo se elige la máscara: se busca la potencia de 2 más pequeña que cubra los hosts (más 2 de red y broadcast).

- 50 hosts + 2 = 52 → 64 (2^6) → 6 bits de host → /26
- 20 hosts + 2 = 22 → 32 (2^5) → 5 bits de host → /27
- 10 hosts + 2 = 12 → 16 (2^4) → 4 bits de host → /28
- 5 hosts + 2 = 7 → 8 (2^3) → 3 bits de host → /29

Regla práctica: ordenad las subredes de **mayor a menor** y asignadlas en ese orden, así no se pisan.

2.5 Privadas, públicas y NAT

No todas las IP son iguales. Hay rangos **privados** que se pueden usar libremente en cualquier red local, pero **no salen a Internet** tal cual:

Rango privado	Máscara
`10.0.0.0` – `10.255.255.255`	/8
`172.16.0.0` – `172.31.255.255`	/12
`192.168.0.0` – `192.168.255.255`	/16

Las **públicas** son únicas en el mundo y las asignan los proveedores.

Entonces, ¿cómo salen a Internet los 30 equipos de una casa con una sola IP pública? Con **NAT** (*Network Address Translation*): el router "traduce" las direcciones privadas a una pública al salir, y apunta en una tabla quién pidió qué (usando puertos) para devolver las respuestas al equipo correcto.

```
PC 192.168.1.10 (privada) ⟶ Router NAT ⟶ 203.0.113.10 (pública) ⟶
Internet
```

Es como una recepcionista de hotel: tú le pides que llame a un restaurante, ella llama desde el teléfono del hotel, y cuando el restaurante responde, ella te pasa la llamada a tu habitación. El restaurante solo conoce el número del hotel.

2.6 IPv6 en 60 segundos

El problema: IPv4 tiene unos **4.300 millones de direcciones**. Parecía muchísimo... hasta que cada persona tuvo móvil, portátil, reloj, bombilla inteligente, nevera conectada y un altavoz que escucha todo lo que decimos.

La solución: **IPv6**, con 128 bits (frente a 32 de IPv4). Cantidad de direcciones: unos **$3{,}4 \times 10^{38}$**. Para hacerse una idea: podríamos darle una dirección a **cada grano de arena** del planeta y sobrarían muchísimas.

Cómo se ve:

```
2001:0db8:0000:0000:0000:ff00:0042:8329     ← completa
2001:db8::ff00:42:8329                       ← abreviada
```

Reglas de abreviación:

1. Se pueden quitar los **ceros a la izquierda** de cada bloque.
2. Una **secuencia** de bloques a cero se puede sustituir por `::` (solo **una vez** por dirección).

Mini-glosario del módulo:

- **Máscara:** separa red de host.
- **CIDR (/n):** notación corta de la máscara.
- **VLSM:** subredes de tamaño variable.

- **NAT:** traducción de direcciones privadas a públicas.

Módulo 3 · Cómo funciona la red

Objetivo del módulo: juntar todo lo visto hasta ahora y seguir, paso a paso, lo que ocurre desde que escribes una dirección en el navegador hasta que ves la página. Al final sabréis explicar qué hace cada pieza (DHCP, DNS, ARP, switch, router, NAT, TCP) y en qué orden entra en escena.

Índice

1. Topología: todos al switch
2. Las dos direcciones: MAC e IP
3. DHCP: cómo consigue tu equipo una IP
4. ARP: ¿quién tiene esa IP?
5. DNS: la agenda de contactos de Internet
6. Puertos: ¿a qué programa va esto?
7. TCP vs. UDP
8. ICMP: ping y traceroute
9. ¿Local o remoto? La decisión del PC
10. El viaje completo de un paquete
11. Cuando algo falla: diagnóstico por pasos
12. Prácticas y retos
13. Mini-glosario

3.1 Topología: todos al switch

En casi cualquier oficina vais a ver la **estrella**: todos los equipos conectados a un switch central.



```
PC 3 ─┤
PC 4 ─┘
```

Ventaja	Si se rompe el cable de un PC, solo se cae ese PC.
Inconveniente	Si se cae el switch, se cae todo el mundo.
Solución	Duplicar los switches importantes (de ahí vienen STP y LACP, que veremos en el módulo 4).

3.2 Las dos direcciones: MAC e IP

Cada equipo tiene **dos direcciones**, y cada una sirve para algo distinto. Es la base de todo el módulo.

	Dirección MAC	Dirección IP
Capa	2 (Enlace)	3 (Red)
Aspecto	`AA:BB:CC:11:22:33`	`192.168.1.10`
Quién la pone	El fabricante (viene "grabada" en la tarjeta de red)	Tú, o el servidor DHCP
Alcance	Solo **dentro de la red local**	Sirve para llegar **a cualquier parte**
Cambia	Casi nunca	Cuando cambias de red

Analogía:

- La **MAC** es tu **DNI**: te identifica a ti, vayas donde vayas.
- La **IP** es tu **dirección postal**: depende de dónde vivas ahora. Si te mudas, cambia.

Un detalle útil: los 3 primeros pares de la MAC identifican **al fabricante** (por ejemplo, para saber si una tarjeta es de Intel, Cisco, etc.). Los otros 3 son el número de serie de esa tarjeta.

Y una MAC especial: `FF:FF:FF:FF:FF:FF` es la MAC de **broadcast**, o sea, "todos".

3.3 DHCP: cómo consigue tu equipo una IP

Cuando conectas un portátil a una red, nadie se sienta a escribirle una IP a mano (al menos, esperemos que no). Eso lo hace el **servidor DHCP** (casi siempre, el propio router).

La conversación se llama DORA:

Paso	Mensaje	Qué dice el equipo / el servidor
D	**Discover**	El equipo grita (broadcast): *"¡Hola! Acabo de llegar, ¿hay algún DHCP por aquí?"*
O	**Offer**	El servidor responde: *"Te ofrezco la 192.168.1.37."*
R	**Request**	El equipo contesta: *"Perfecto, la quiero."*
A	**Acknowledge**	El servidor confirma: *"Es tuya durante 24 h."*

Qué entrega el DHCP (no es solo la IP):

- Dirección IP
- Máscara de subred
- **Puerta de enlace** (gateway)
- Servidores **DNS**
- **Tiempo de concesión** (cuánto tiempo puedes quedártela)

Es como llegar a un hotel: la recepción te asigna una habitación (IP), te dice cuánto tiempo puedes usarla (concesión), dónde está la salida (puerta de enlace) y dónde está la guía telefónica (DNS).

Mini-curiosidad: si tu equipo no consigue IP del DHCP, Windows se pone una a sí mismo del tipo `169.254.x.x` (APIPA). Si veis esa dirección en `ipconfig`, significa: *"nadie me ha dado IP"*. Pista inmediata de que el DHCP no responde (o el cable no llega).

3.4 ARP: ¿quién tiene esa IP?

Aquí está la pregunta clave: si yo sé la **IP** del destino, pero las tramas dentro de la LAN viajan por **MAC**, ¿cómo averiguo la MAC?

Con **ARP** (*Address Resolution Protocol*):

1. **PC-A** quiere hablar con `192.168.1.20`, pero no conoce su MAC.
2. Grita a todos (**broadcast**, destino `FF:FF:FF:FF:FF:FF`): *"¿Quién tiene la IP 192.168.1.20? Que me diga su MAC."*
3. Todos lo escuchan, pero **solo PC-B** se da por aludido y responde (**unicast**): *"¡Soy yo! Mi MAC es BB:BB."*
4. PC-A apunta la respuesta en su **tabla ARP** (una especie de "chuleta") para no preguntar de nuevo durante un rato.

```
PC-A: "¿Quién tiene la .20?"  ──broadcast──►  TODOS
PC-B: "¡Soy yo! MAC BB:BB"    ◄──respuesta──  PC-B
```

Para ver la tabla ARP de tu equipo:

```
arp -a
```

Una pregunta que siempre sale: ¿y si el destino está en *otra* red?
Entonces el PC **no pregunta por la MAC del destino final** (no podría: está en otra LAN). Pregunta por la **MAC de su puerta de enlace** (el router) y le entrega la trama a él. El router se encargará del resto.

Analogía: entras en una sala llena de gente y gritas *"¿Quién es Antonio?"*. Todos te miran, pero solo Antonio levanta la mano. A partir de ahí, ya sabes qué cara tiene Antonio y no hace falta gritar más.

3.5 DNS: la agenda de contactos de Internet

Los humanos recordamos **nombres** (`google.com`); las redes necesitan **IP**. El **DNS** (*Domain Name System*) es el servicio que **traduce nombres a direcciones IP**.

Es como la **agenda de tu móvil**: tú pulsas en "Mamá", y el teléfono sabe qué número marcar. Nadie se acuerda de números de teléfono; tampoco de IP.

Cómo funciona (versión corta):

1. Escribes `www.ejemplo.com` en el navegador.
2. Tu PC mira primero en su **caché** (¿ya lo he consultado hace poco?).
3. Si no está, pregunta al **servidor DNS** que le dio el DHCP.

4. Ese servidor, si no lo sabe, va preguntando por Internet hasta encontrar la respuesta, y te la devuelve.
5. Ya tienes la IP. Ahora sí, el navegador puede conectarse.

Tipos de registros más comunes:

Registro	Para qué sirve	Ejemplo
A	Nombre → IPv4	`ejemplo.com → 203.0.113.10`
AAAA	Nombre → IPv6	`ejemplo.com → 2001:db8::10`
CNAME	Un nombre es "alias" de otro	`www → ejemplo.com`
MX	Dónde está el servidor de correo	`correo.ejemplo.com`

Comandos para probarlo:

```
nslookup www.google.com        (Windows / Linux / Mac)
dig www.google.com             (Linux / Mac)
```

Truco de diagnóstico de oro:

- `ping 8.8.8.8` funciona **pero** `ping google.com` falla → el problema es el **DNS**.
- Ninguno de los dos funciona → el problema está **antes** (cable, IP, puerta de enlace...).

3.6 Puertos: ¿a qué programa va esto?

Una IP identifica **un equipo**. Pero ese equipo tiene muchos programas funcionando a la vez: el navegador, el correo, el chat... ¿Cómo sabe a cuál entregar cada mensaje?

Con los **puertos** (capa 4, Transporte). Si la IP es la **dirección del edificio**, el puerto es **el número de la puerta del piso**.

Puertos que conviene conocer:

Puerto	Servicio	Para qué
22	SSH	Administración remota segura
25	SMTP	Envío de correo

Puerto	Servicio	Para qué
53	DNS	Traducir nombres
67 / 68	DHCP	Repartir IP
80	HTTP	Web (sin cifrar)
443	HTTPS	Web (cifrada)

Cuando tu navegador visita una web:

- **Destino:** `IP del servidor : 443` (el servidor atiende ahí).
- **Origen:** `tu IP : un puerto alto aleatorio` (por ejemplo, el 52114), para que el servidor sepa a qué pestaña devolver la respuesta.

A la combinación **IP + puerto** se le llama **socket**.

Y ahora el vínculo con el módulo 2: así funciona el **NAT**. El router apunta en una tabla *"el puerto 52114 de 192.168.1.10 salió como el puerto 40001 de la IP pública"*, y cuando vuelve la respuesta, deshace el cambio.

3.7 TCP vs. UDP

En la capa de Transporte hay **dos formas** de enviar datos:

	TCP	UDP
Estilo	**Fiable**: confirma, reenvía lo perdido, ordena	**Rápido**: lo envía y a otra cosa
Analogía	**Burofax** con acuse de recibo	**Postal** sin remite ni confirmación
Conexión previa	Sí (el *handshake*)	No
Se usa en	Web, correo, descargas, SSH	Videollamadas, juegos, DNS, streaming
Si se pierde un paquete	Se reenvía	Se pierde y ya

¿Por qué existe el "poco fiable"? Porque a veces la velocidad importa más que la perfección. En una videollamada, si se pierde un trocito de audio de hace un segundo, no sirve de nada reenviarlo: ya habéis seguido hablando.

El saludo de TCP (three-way handshake)

Antes de intercambiar datos, TCP "se presenta":

```
Cliente ——— SYN ———————▶  Servidor    "¿Hablamos?"
Cliente ◀——— SYN + ACK ———  Servidor    "Sí, hablamos. ¿Me oyes?"
Cliente ——— ACK ———————▶  Servidor    "Te oigo. Empezamos."
```

Es como un **llamar por teléfono**: "¿Diga?" - "¡Hola, soy yo!" - "¡Ah, hola!". Hasta que no hay este saludo, nadie suelta información importante.

3.8 ICMP: ping y traceroute

ICMP es el protocolo "de mantenimiento" de la red: sirve para **avisar y comprobar**. No transporta datos de usuario, pero es oro para diagnosticar.

`ping`: ¿estás ahí?

Envía un mensaje (*Echo Request*) y espera respuesta (*Echo Reply*). Te dice **si llegas** y **cuánto tarda** (latencia, en milisegundos).

```
ping 192.168.1.1
```

Qué te cuenta:

- **Responde rápido** (1-5 ms) → llegas bien.
- **Tiempo agotado** → no hay respuesta (o alguien la bloquea).
- **Destino inaccesible** → un router te dice que no sabe cómo llegar.

> ⚠️ **Ojo:** que no responda al ping **no siempre** significa que esté caído. Muchos firewalls bloquean el ping a propósito.

`traceroute` (Linux/Mac) / `tracert` (Windows): ¿por dónde pasas?

Te enseña **cada router** (salto) por el que pasa tu paquete hasta el destino.

```
tracert www.google.com        (Windows)
traceroute www.google.com     (Linux / Mac)
```

¿Cómo lo hace? Con el TTL. Cada paquete IP lleva un contador llamado **TTL** (*Time To Live*) que **baja una unidad en cada router**. Cuando llega a 0, el router lo tira y avisa al origen.

Traceroute lo aprovecha: envía un paquete con TTL=1 (muere en el primer router, que se "delata"), luego TTL=2 (muere en el segundo), y así sucesivamente.

Para qué sirve el TTL en general: evita que un paquete dé vueltas **eternamente** si hay un bucle de routing. (Recuerda del módulo 4: las tramas Ethernet **no tienen** TTL, por eso los bucles de switches son tan peligrosos.)

3.9 ¿Local o remoto? La decisión del PC

Antes de enviar nada, el PC se hace **una pregunta**: *"¿El destino está en mi misma red o en otra?"* Y lo averigua con su **IP y su máscara**.

Ejemplo:

- Mi PC: `192.168.1.10/24` → mi red es `192.168.1.0/24` (de .1 a .254).
- Destino A: `192.168.1.50` → **misma red** → ARP directo al destino.
- Destino B: `8.8.8.8` → **otra red** → se lo mando a la **puerta de enlace**.

```
¿Destino dentro de mi red?
   ├─ SÍ → ARP del destino  → trama directa a su MAC
   └─ NO → ARP de la puerta de enlace → trama a la MAC del router
```

Detalle importante: en ambos casos la **IP de destino** del paquete es la del destino final. Lo que cambia es **a qué MAC** se entrega la trama en el primer salto.

> 😄 **Historia de pasillo:** un técnico puso mal la máscara en un PC (`255.255.255.128` en vez de `255.255.255.0`). El PC "creía" que media oficina estaba en otra red y mandaba todo a la puerta de enlace, y esta no sabía qué hacer. Síntoma: *"Puedo ver a unos compañeros, pero a otros no"*. La máscara: pequeña, silenciosa y letal.

> 🤔 **Para lanzar a clase:** un PC tiene la IP `192.168.1.10/24` y quiere hablar con `192.168.2.10`. ¿Va directo o a la puerta de enlace? ¿Por qué?
> *(Respuesta: a la puerta de enlace; `192.168.2.x` no está en su red /24.)*

3.10 El viaje completo de un paquete

Ahora lo juntamos todo. Quieres abrir `www.ejemplo.com`:

```
Tu PC → Switch → Router (NAT) → Internet → Servidor
```

Escenario

- Tu PC: `192.168.1.10`, MAC `AA:AA`
- Router (puerta de enlace): `192.168.1.1`, MAC `RR:RR`
- IP pública del router: `203.0.113.10`
- Servidor web: `198.51.100.20`

Paso a paso

Paso 0 · (Ya pasó antes) DHCP. Al conectarte te dieron IP, máscara, puerta de enlace y DNS.

Paso 1 · DNS. Tu PC pregunta: *"¿Cuál es la IP de www.ejemplo.com?"* Respuesta: `198.51.100.20`.

Paso 2 · ¿Local o remoto? `198.51.100.20` no está en `192.168.1.0/24` → hay que mandarlo a la puerta de enlace.

Paso 3 · ARP. Tu PC pregunta: *"¿Quién tiene la 192.168.1.1?"* El router responde con su MAC `RR:RR`.

Paso 4 · TCP handshake. Tu PC envía un SYN al puerto 443 del servidor. (El SYN viaja con todo el proceso de abajo, y la respuesta SYN-ACK vuelve por el camino inverso; luego tu PC manda el ACK.)

Paso 5 · Se arma el paquete (encapsulación):

Capa	Qué se añade
Transporte	Puerto origen 52114 → puerto destino 443
Red	IP origen `192.168.1.10` → IP destino `198.51.100.20`
Enlace	MAC origen `AA:AA` → MAC destino `RR:RR`

Paso 6 · El switch. Recibe la trama, mira la MAC destino `RR:RR` en su tabla y la envía **solo por el puerto donde está el router**.

Paso 7 · El router.

1. Quita la trama, mira la **IP destino** y consulta su **tabla de enrutamiento**.
2. Aplica **NAT**: cambia la IP origen `192.168.1.10` por `203.0.113.10` (y apunta en su tabla quién pidió qué).
3. **Resta 1 al TTL.**
4. Crea una **trama nueva** con las MAC del siguiente salto.

Paso 8 · Internet. Una cadena de routers repite el proceso: cada uno mira la IP destino, elige el siguiente salto, resta 1 al TTL y reescribe la trama.

Paso 9 · El servidor. Recibe, responde, y **el camino se repite al revés**. Al llegar a tu router, este **deshace el NAT** (consulta su tabla) y se lo entrega a tu PC.

Lo que cambia y lo que no cambia

	¿Cambia en el camino?
IP origen / destino	**No** (salvo por el NAT al salir de tu red)
MAC origen / destino	**Sí, en cada salto**
TTL	Baja 1 en cada router
Puertos	Se mantienen (el NAT puede cambiar el de origen)

> 💡 **Idea clave para llevarse a casa:** la **IP** es la **dirección del destino final**; la **MAC** es solo **"el siguiente paso"**. Como un viaje en tren con transbordos: tu destino final no cambia, pero en cada estación coges un tren distinto.

> 😄 **Todo esto ocurre en milisegundos, cada vez que abres una web.** Y luego nos quejamos si tarda 2 segundos. Un poco de respeto al paquete.

3.11 Cuando algo falla: diagnóstico por pasos

Con todo lo del módulo, ya tenemos un **método** para investigar el clásico *"no tengo Internet"*. La regla: **de abajo hacia arriba**.

Paso	Pregunta	Cómo comprobarlo	Si falla...
1	¿Hay conexión física?	Luz del puerto, wifi conectado	Capa 1: cable, wifi, puerto
2	¿Tengo IP válida?	`ipconfig` / `ip a`	¿Sale `169.254.x.x`? → DHCP
3	¿Llego a mi puerta de enlace?	`ping 192.168.1.1`	Problema en la LAN (switch, VLAN, cable)
4	¿Llego a Internet por IP?	`ping 8.8.8.8`	Problema en el router / ISP / ruta
5	¿Resuelvo nombres?	`ping google.com` o `nslookup`	Problema de DNS
6	¿Funciona la aplicación?	Abrir la web / servicio	Servidor caído, firewall, puerto bloqueado

Resumen visual de la lógica:

```
¿ping a la puerta de enlace?  NO → problema LOCAL (cable, IP, switch)
            | SÍ
¿ping a 8.8.8.8?              NO → problema del ROUTER o del ISP
            | SÍ
¿ping a google.com?           NO → problema de DNS
            | SÍ
¿abre la web?                 NO → problema de la APLICACIÓN
```

> 😄 **Historia de pasillo:** una usuaria ficticia llamó: *"Internet no funciona"*. El técnico, siguiendo el método, comprobó cada paso. Todo bien. Al final descubrió que lo que no funcionaba era **la web concreta** que ella quería abrir, que estaba caída para todo el mundo. Lección: antes de culpar a tu red, comprueba si el servidor existe. (Hay webs para ello: se llaman *"is it down"*, y salvan carreras.)

3.12 Prácticas y retos

Práctica 1 · Mi propio equipo (10 min)

1. Abrid un terminal y anotad vuestra **IP, máscara, puerta de enlace, DNS y MAC**.

2. Calculad cuál es vuestra **dirección de red** y vuestro **broadcast**.

Práctica 2 · ARP en acción (10 min)

1. Ejecutad `arp -a` y anotad cuántas entradas hay.
2. Haced `ping` a un compañero.
3. Volved a ejecutar `arp -a`. **¿Qué ha cambiado?**
4. Identificad la entrada de vuestra **puerta de enlace**. ¿Reconocéis su MAC?

Práctica 3 · DNS (10 min)

1. `nslookup www.google.com` → ¿qué IP devuelve? ¿Y si lo repetís? ¿Cambia?
2. Probad con `nslookup` de una web de vuestra empresa o centro.
3. Comparad `ping 8.8.8.8` con `ping google.com`. ¿Cuál tarda más en arrancar y por qué?

Práctica 4 · Seguir el camino (10 min)

1. `tracert` (o `traceroute`) a una web conocida.
2. **¿Cuántos saltos hay?** ¿Veis algún salto con latencia mucho más alta que los demás?
3. Probad a una web de otro continente y comparad.

> 🏁 **Reto de diagnóstico:** el profesor da un síntoma, y el grupo debe decir **qué comprobaría y en qué orden**:
>
> 1. *"Veo a mis compañeros, pero no tengo Internet."*
> 2. *"Puedo hacer ping a Google por IP, pero no abre ninguna web por nombre."*
> 3. *"Mi IP empieza por 169.254."*
> 4. *"Algunas webs abren y otras no."*
>
> **Pistas:** (1) router/ISP/ruta; (2) DNS; (3) DHCP; (4) puede ser DNS parcial, firewall, o la propia web.

3.13 Mini-glosario

Término	Significado
MAC	Dirección física de la tarjeta de red (capa 2).
IP	Dirección lógica del equipo en la red (capa 3).

Término	Significado
DHCP	Servicio que reparte IP, máscara, gateway y DNS automáticamente.
DORA	Los 4 pasos de DHCP: Discover, Offer, Request, Acknowledge.
ARP	Protocolo para averiguar la MAC a partir de una IP.
Tabla ARP	Chuleta donde el equipo apunta las parejas IP-MAC aprendidas.
DNS	Servicio que traduce nombres a direcciones IP.
Puerto	Número que identifica el programa destino dentro de un equipo (capa 4).
Socket	Combinación IP + puerto.
TCP	Transporte fiable, con conexión y confirmaciones.
UDP	Transporte rápido, sin confirmaciones.
Handshake	Saludo de tres pasos (SYN, SYN-ACK, ACK) que abre una conexión TCP.
ICMP	Protocolo de diagnóstico (lo que usa `ping`).
TTL	Contador que baja 1 por router y evita que un paquete dé vueltas eternamente.
Puerta de enlace	El router por el que salgo hacia otras redes.
Broadcast	Mensaje para todos.
Unicast	Mensaje para uno solo.
NAT	Traducción de direcciones privadas a públicas (usando puertos).
Latencia	Lo que tarda un paquete en ir y volver (en milisegundos).

Para llevarse a casa

1. La **IP** es el destino final; la **MAC** es el siguiente salto.
2. Antes de nada, el PC se pregunta: **¿está el destino en mi red o no?**
3. **DHCP** te da la IP, **ARP** averigua la MAC, **DNS** traduce el nombre.
4. **Puertos** dicen *a qué programa* va el dato; **TCP/UDP** dicen *cómo*.

5. Ante un fallo: **de abajo hacia arriba**, y casi siempre es la capa 1... o el DNS. 😉

Módulo 4 · Switching

Objetivo del módulo: entender cómo piensa un switch, cómo se segmenta una red con VLAN, cómo se evitan los bucles y cómo se diagnostica cuando algo no va.

4.1 Cómo aprende un switch

Un switch **no viene configurado con las direcciones de los equipos**. Las aprende solo, mirando las tramas que pasan:

1. Llega una trama por el **puerto 1**, con origen `AA:AA`. El switch apunta: *"AA:AA está en el puerto 1."*
2. Si el destino **ya está** en su tabla → lo manda solo a ese puerto.
3. Si el destino **no está** → lo manda a **todos los puertos** menos por el que entró (*flooding*), hasta que el destino responda y se aprenda su ubicación.

Tabla de direcciones MAC (ejemplo):

MAC	Puerto
AA:AA	1
BB:BB	2

Para verla en un switch Cisco: `show mac address-table`.

> 😄 **Analogía:** el switch es como un portero nuevo en un edificio. El primer día no sabe quién vive dónde. Pero cada vez que alguien entra diciendo "vengo del 3ºB", él lo apunta en su libreta. A la semana lo sabe todo de todos. (Y, como todo buen portero, nunca olvida... hasta que lo reinicias.)

4.2 VLAN: pisos independientes

Una red física puede **partirse en redes lógicas** independientes. Eso son las **VLAN** (*Virtual LAN*).

Imagina un edificio: el mismo edificio, el mismo ascensor y la misma escalera, pero cada piso es independiente. Los vecinos del 1º no entran en el 2º.

```
          ┌─ VLAN 10 · Administración: PC-A, PC-B
[SWITCH] ─┤
          └─ VLAN 20 · Invitados:      PC-C, PC-D
```

Por qué se usan:

- **Seguridad:** que los invitados no puedan husmear en el servidor de nóminas.
- **Rendimiento:** menos broadcast circulando por todas partes.
- **Orden:** organizar por departamentos y no por la forma del cableado.

Por defecto, **equipos en VLAN distintas no se ven entre sí**, aunque estén en el mismo switch físico. Para que se hablen, hace falta un router (lo veremos en el módulo de routing).

> 🤔 **Para lanzar a clase:** ¿por qué separar *Invitados* de *Administración* aunque compartan el mismo switch físico?

> 😄 **Historia de pasillo:** una empresa ficticia tenía una sola red para todo. El día que un invitado conectó su portátil, "descubrió" por casualidad la impresora de Dirección... y le mandó 400 copias de su CV. Con VLAN, eso no habría pasado. (Aunque a lo mejor lo contrataron.)

4.3 Access vs. Trunk

Un puerto de switch puede funcionar de dos modos:

	Access	Trunk
VLAN	**Una única** VLAN	**Varias** VLAN a la vez
Tráfico	**Sin etiquetar**	**Etiquetado** (802.1Q)
Para qué	PC, impresoras, cámaras...	Unir switches entre sí (o switch con router)

Analogía: un puerto *access* es una puerta de una vivienda: da a un solo piso. Un *trunk* es el ascensor: sirve a todos los pisos y por eso necesita saber a cuál va cada pasajero.

4.4 802.1Q: la etiqueta viajera

Cuando una trama viaja por un trunk, necesita llevar escrito **a qué VLAN pertenece**. Para eso se le añade una **etiqueta 802.1Q** (4 bytes):

```
Ethernet · 802.1Q (VID 20) · Datos · FCS
```

- El **VID** (VLAN ID) tiene 12 bits: hay hasta **4094 VLAN** utilizables.
- La etiqueta **se añade** al entrar al trunk y **se quita** al salir por el puerto access de destino. El PC final nunca la ve.

> 🏁 **Reto:** ¿dónde se añade la etiqueta y dónde se quita? ¿Qué pasa si llega a un puerto access?
> 💡 **Chuleta:** se añade al entrar al trunk y se quita al salir por el puerto access de destino.

Un detalle para curiosos: la VLAN nativa. En un trunk, una VLAN (por defecto la 1) viaja *sin* etiqueta. Si los dos extremos no se ponen de acuerdo en cuál es, ocurren cosas raras. Buena práctica: que ambos lados coincidan y no usar la VLAN 1 para nada serio.

4.5 Spanning Tree Protocol (STP): evitando bucles

Para tener redundancia se ponen **dos caminos** entre switches. Parece buena idea, pero genera un problema serio: **un bucle**.

En una LAN, un broadcast que entra en un bucle **no muere nunca**: da vueltas y vueltas, y se multiplica. Es lo que se llama una **tormenta de broadcast** (*broadcast storm*), y puede tumbar una red entera en segundos. A diferencia de los paquetes IP, las tramas Ethernet no tienen "fecha de caducidad" (no hay TTL).

La solución: STP. Los switches se ponen de acuerdo para:

1. Elegir un **switch raíz** (*root bridge*), el que tenga el menor identificador.
2. Calcular el mejor camino hacia la raíz.
3. **Bloquear lógicamente** los enlaces sobrantes, dejándolos "dormidos". Si el principal falla, el bloqueado despierta y toma el relevo.

```
SW1 ═══════════ SW2
 ╚═══(BLOQUEADO)═══╝      ← enlace redundante, apagado lógicamente
```

(En la práctica se usa RSTP, la versión rápida, que reacciona en segundos en vez de casi un minuto.)

> 😄 **Historia de pasillo (de las que enseñan):** un becario, con la mejor intención del mundo, "conectó el cable que sobraba" entre dos switches de la oficina. Sin STP. Dos minutos después, la red no es que fuera lenta: era *inexistente*. Los LED de todos los puertos parpadeaban como una discoteca. Lo llamamos "la noche de los bucles vivientes". El becario sigue en la empresa... y ahora **revisa los cables antes de enchufar**.

4.6 LACP y EtherChannel: sumar cables

Si un cable se queda corto de velocidad, se pueden **agrupar varios en uno solo lógico**: eso es **EtherChannel** (Cisco) o **LAG / LACP** (el estándar, 802.3ad / 802.1AX).

```
SW1 ═══ 2 enlaces = 1 canal ═══ SW2
```

Ventajas:

- **Más ancho de banda** total (dos cables de 1 Gbps ≈ un "canal" de 2 Gbps).
- **Redundancia**: si un cable se cae, el canal sigue funcionando por el otro.
- STP lo ve como **un solo enlace**, así que no bloquea nada.

Matiz importante: el tráfico se reparte *por conversación* (flujo), no paquete a paquete. Así, una única descarga enorme no se acelera por tener dos cables; lo que mejora es el conjunto de muchas conversaciones a la vez.

> 🤔 **Para lanzar a clase:** si se cae uno de los dos cables del canal, ¿qué pasa con el tráfico?
> *(Respuesta: sigue funcionando por el cable que queda, con menos capacidad. Como una carretera de dos carriles con uno cortado: se llega, pero con retenciones.)*
>
> 💡 **Chuleta:** comando clave: `show etherchannel summary`.

4.7 CDP/LLDP + un caso real

¿Cómo sabe un switch quién tiene al lado? Con protocolos de descubrimiento de vecinos:

- **CDP** (*Cisco Discovery Protocol*): propietario de Cisco.

- **LLDP** (*Link Layer Discovery Protocol*): el estándar abierto.

Con ellos puedes ver, desde un switch, **qué hay conectado en cada puerto** (nombre del vecino, su puerto, modelo...). Es como preguntar a cada puerta: *"¿Quién vive al otro lado?"*

```
SW1 → Puerto 1 → SW2 Gi1/0/1
```

> 😄 **Utilidad real:** cuando heredas una instalación sin documentación (y créeme, **siempre** se hereda sin documentación), estos comandos son tu mapa del tesoro.

El caso real: "los PC de VLAN 20 no se ven"

> **Fallo típico:** la VLAN 20 no está permitida en el trunk, así que el tráfico de esa VLAN se pierde.

Síntomas: dos switches, un trunk entre ellos, y los PC de la VLAN 20 que están en switches distintos no se ven entre sí. Los de la VLAN 10 sí.

Cómo diagnosticar (de más simple a más complejo):

1. ¿Está el trunk **activo** (capa 1 y 2)?
2. ¿La **VLAN 20 existe** en ambos switches?
3. ¿Está la VLAN 20 **permitida en el trunk** (*allowed VLANs*)?
4. ¿Coincide la **VLAN nativa** en los dos extremos?
5. ¿Los puertos de los PC están en la **VLAN correcta**?

> 🏁 **Reto:** diagnosticad: dos switches, un trunk, y los PC de VLAN 20 no se ven. ¿Qué comprobaríais primero?
> 💡 **Chuleta:** `show cdp neighbors` / `show lldp neighbors` · `show interfaces trunk`.

4.8 PoE: Power over Ethernet

Con **PoE**, el mismo cable de red lleva **datos y energía**. Sin enchufe.

```
Switch PoE ═══ datos + energía ═══▶ AP / Cámara (sin enchufe)
```

Estándares habituales:

Estándar	Potencia aproximada por puerto
802.3af (PoE)	hasta ~15 W
802.3at (PoE+)	hasta ~30 W
802.3bt (PoE++)	hasta ~60-90 W

Para qué se usa: puntos de acceso Wi-Fi, cámaras IP, teléfonos IP, relojes de fichar, etc. Ventaja enorme: puedes poner una cámara en lo alto de un poste donde no hay ni un enchufe.

Ojo con el presupuesto: un switch PoE tiene una **potencia total limitada**. Si le cuelgas más cacharros de los que aguanta, algunos se quedan sin corriente.

> 🤔 **Para lanzar a clase:** ¿qué dispositivos del aula imagináis alimentados por PoE en vez de con un enchufe?

> 😄 **Historia de pasillo:** el técnico pidió "20 cámaras en PoE" y le llegó un switch con presupuesto para 12. Las cámaras 13 a 20 permanecieron ciegas con dignidad. Moraleja: **mira los vatios, no solo los puertos.**

4.9 Port Mirroring (SPAN): copiar el tráfico para vigilarlo

A veces necesitas **ver** qué pasa por un puerto sin tocarlo. Para eso existe el **port mirroring** (SPAN en Cisco): el switch **copia** el tráfico de un puerto a otro "puerto espejo", donde conectas un analizador (por ejemplo, Wireshark).

```
Puerto real ──► SWITCH ──► Puerto espejo (copia del tráfico) ──► Analizador
```

- No interfiere con el tráfico original.
- El equipo vigilado **ni se entera**.

> 🏁 **Reto:** ¿para qué querría un administrador "escuchar" el tráfico de otro puerto sin tocarlo? Pensad 2 motivos.
> 💡 **Chuleta:** análisis de tráfico, detección de intrusos, diagnóstico de una avería.

> 😄 **Importante (y legal):** esto se usa en **tus propias redes y con autorización**. Escuchar el tráfico ajeno sin permiso es un problema serio, y no del tipo "ups". Poder hacerlo no equivale a deber hacerlo.

Mini-glosario del módulo:

- **VLAN:** red lógica independiente.
- **Access / Trunk:** puerto de una VLAN / puerto de varias VLAN.
- **802.1Q:** estándar de etiquetado de VLAN.
- **STP:** evita bucles bloqueando enlaces redundantes.
- **LACP / EtherChannel:** varios cables como uno.
- **PoE:** datos y energía por el mismo cable.
- **SPAN:** copia del tráfico para analizarlo.

Módulo 5 · Routing

Objetivo del módulo: entender cómo decide un router, configurar rutas a mano, conocer los protocolos dinámicos, filtrar tráfico y conseguir que dos VLAN se hablen.

5.1 ¿Qué hace un router?

Un switch trabaja **dentro** de una red. Un **router conecta redes distintas**.

```
PC → Switch → Router → Servidor (en otra red)
```

Su trabajo:

1. Recibe un paquete.
2. Mira la **IP de destino**.
3. Consulta su **tabla de enrutamiento**.
4. Decide por dónde reenviarlo (el **siguiente salto**).
5. **Reescribe la trama** (nuevas MAC) y lo envía.

> 💡 **En una frase:** decide el mejor camino y reescribe la trama en cada salto.

> 😄 **Analogía:** el router es un cruce de carreteras con un guardia de tráfico muy listo: no conoce el destino final de cada coche, pero sabe a qué carretera lo debe mandar para que *se acerque*. Y así, cruce a cruce, todos llegan (casi todos).

5.2 La tabla de enrutamiento

Es **el GPS del router**. Cada línea responde a: *"Si el destino está en esta red, mándalo por aquí."*

Destino	Next-hop	Origen
`10.10.10.0/24`	conectada	**C**
`10.20.0.0/16`	`10.0.0.2`	**O**
`0.0.0.0/0`	`10.0.0.1`	**S***

Qué significan los orígenes:

- **C (Connected):** red directamente conectada al router.
- **S (Static):** ruta puesta a mano.
- **O (OSPF):** aprendida por el protocolo OSPF.
- **S*** o **ruta por defecto (`0.0.0.0/0`):** *"Si no sé a dónde mandarlo... mándalo por aquí."* Es la salida hacia Internet. Es la respuesta del router cuando dice "yo qué sé, pregúntale al jefe".

Para verla en un equipo Cisco: `show ip route`. En Linux: `ip route`.

5.3 La ruta más precisa gana (longest prefix match)

¿Qué pasa si **varias rutas** encajan con el mismo destino? **Gana la más específica**, es decir, la que tiene la **máscara más larga**.

Ruta candidata	Prefijo
`10.0.0.0/8`	menos específica
`10.20.0.0/16`	más específica
`10.20.30.0/24`	**¡esta gana!**

Ejemplo: un paquete para `10.20.30.55` encaja con las tres rutas. El router elige `/24`, la más concreta.

Analogía: si preguntas por "calle Larga, nº 8, 3ºB", prefiere la información que llega hasta el piso, antes que la que solo dice "una calle de la ciudad". Cuanto más detalle, mejor.

5.4 Rutas estáticas

Una **ruta estática** se escribe **a mano**. Le dices al router: *"Para llegar a esta red, ve por ese vecino."*

```
R1 ——— R2 ——— R3
```

Ejemplo (en R1, para llegar a la red `10.20.0.0/16` a través de R2):

```
ip route 10.20.0.0 255.255.0.0 10.0.0.2
```

Ventajas	Inconvenientes
Sencillas y predecibles	Si cambia la red, hay que cambiarlas a mano
No gastan recursos	En redes grandes se vuelven una pesadilla
Ideales para redes pequeñas o rutas por defecto	Si olvidas una, hay un agujero negro

Importante: la comunicación es **bidireccional**. Si R1 sabe llegar a R3 pero R3 no sabe volver, el ping **no funciona**. Una de las razones más típicas de "llega pero no vuelve".

> 🏁 **Reto:** R1 y R3 no se ven todavía. ¿Qué ruta estática le falta a cada router?

> 😄 **Historia de pasillo:** un técnico puso la ruta en R1 y estuvo una hora preguntándose por qué el ping no respondía. El paquete llegaba, la respuesta no sabía volver. *"Es como mandar una carta sin remitente: llega, pero nadie sabe a quién contestar."*

5.5 RIP vs. OSPF: protocolos dinámicos

En vez de escribir todas las rutas a mano, los routers pueden **hablar entre sí y aprenderlas**. Eso son los **protocolos de enrutamiento dinámico**.

	RIP	OSPF
Métrica	Nº de saltos	Coste (basado en ancho de banda)
Velocidad	Sencillo, pero lento en reaccionar	Rápido y escalable
Tamaño ideal	Redes muy pequeñas	Redes medianas y grandes
Límite	Máximo 15 saltos	Muy escalable (con áreas)

RIP piensa: *"El mejor camino es el que tiene menos saltos"* (aunque sea un camino de cabras). **OSPF** piensa: *"El mejor camino es el más rápido"* (aunque tenga más saltos).

Ejemplo: para ir de A a B, un camino pasa por 2 routers pero con un enlace lentísimo, y otro pasa por 4 routers con enlaces rapidísimos. RIP elige el primero (menos saltos); OSPF elige el segundo (más ancho de banda).

> 😄 **Moraleja:** RIP es el amigo que dice "por aquí es más corto" y te mete por un camino de tierra. OSPF mira el mapa de tráfico antes de salir.

5.6 Listas de control de acceso (ACL)

Una **ACL** es una **lista de reglas** que dice **quién puede pasar y quién no**. Se evalúa **de arriba abajo** y la primera regla que encaja, manda.

```
Invitados ──► [ ACL 🚫 ] ──► Servidores
```

Ejemplo de ACL (sintaxis Cisco, simplificada):

```
ip access-list extended INVITADOS-A-SERVIDORES
 deny   ip 192.168.20.0 0.0.0.255 192.168.30.0 0.0.0.255
 permit ip any any
```

Traducción: "A los de la red de invitados **les prohíbo** hablar con la red de servidores. Todo lo demás, adelante."

Dos cosas que hay que recordar siempre:

1. **Orden:** la primera coincidencia gana. Una regla mal colocada puede anular todo lo demás.
2. **Al final de toda ACL hay un `deny any` implícito.** Si no pones un `permit` final, **todo lo no permitido se bloquea**.

> 🤔 **Para lanzar a clase:** ¿se os ocurre un caso real donde interese bloquear tráfico entre dos partes de la misma red?

> 😄 **Historia de pasillo:** un administrador escribió una ACL perfecta... y al aplicarla se quedó **sin acceso al propio router** que estaba configurando. Hay una regla de oro: *"Antes de aplicar una ACL remota, asegúrate de que no te estás cerrando tú la puerta."* Y si hace falta, ponle un reinicio programado de seguridad.

5.7 Router-on-a-Stick: uniendo switching y routing

Del módulo anterior: **VLAN distintas no se ven entre sí.** Para que lo hagan, hace falta un **router** (o un switch de capa 3).

El truco, cuando solo tenemos **un cable** entre el switch y el router: usar un **trunk 802.1Q** y dividir esa interfaz física en **subinterfaces**, una por VLAN, cada una con su **puerta de enlace**.

```
          trunk 802.1Q
[SWITCH] ═══════════════ [ROUTER]
                          ├─ subinterfaz .10 → VLAN 10 (192.168.10.1)
                          └─ subinterfaz .20 → VLAN 20 (192.168.20.1)
```

Configuración de ejemplo (Cisco):

```
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
!
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
```

Y en los PC: cada uno usa como **puerta de enlace** la IP de la subinterfaz de su VLAN (los de la VLAN 10 usan `192.168.10.1`, los de la VLAN 20 usan `192.168.20.1`).

Por qué se llama "on a stick" (en un palo): el router tiene **un solo cable** (el "palo") por el que entra y sale el tráfico de todas las VLAN. Funciona muy bien en redes pequeñas, aunque ese único cable puede ser un cuello de botella.

> 🏁 **Reto:** retomad la red con VLAN del tema anterior: ¿cómo conseguiríais que VLAN 10 y VLAN 20 se hablen?

> 😄 **Para recordarlo:** el router es un traductor de un congreso internacional que está en una sola cabina, atendiendo a varios idiomas por el mismo micrófono. Funciona, pero como la cola sea larga, se forma un atasco.

Mini-glosario del módulo:

- **Tabla de enrutamiento:** el "GPS" del router.
- **Next-hop:** siguiente router en el camino.
- **Ruta por defecto (`0.0.0.0/0`):** "por si acaso" hacia Internet.
- **RIP / OSPF:** protocolos dinámicos (saltos / ancho de banda).
- **ACL:** reglas para permitir o bloquear.
- **Router-on-a-stick:** un router con subinterfaces para conectar VLAN.

Módulo 6 · Zabbix

Objetivo del módulo: pasar de "apagar fuegos" a **evitarlos**. Entender cómo funciona Zabbix y montar una monitorización básica de una red.

6.1 Por qué monitorizar: de reactivo a proactivo

	Sin monitorización	Con Zabbix
Cómo te enteras	Cuando **llama el usuario**	**Antes** de que llame
Estado de ánimo	😱	😎
Tiempo de reacción	Variable (muy variable)	Rápido
Historial	"Creo que ayer también fue lento..."	Datos y gráficas

Sin monitorización: el jefe te llama enfadado, y entonces te enteras de que la red está caída desde las 8:00. Son las 11:30.
Con Zabbix: te llega un aviso a las 8:01 y a las 8:15 ya está resuelto, y el jefe nunca se entera.
El mejor técnico es el que no se nota.

> 😄 **Historia de pasillo:** una empresa ficticia descubrió que un disco estaba al 100% cuando el servidor dejó de funcionar. El disco llevaba **meses** subiendo poco a poco: 60%... 75%... 90%... Con monitorización, habría saltado una alerta a tiempo. Sin ella, saltó el director.

6.2 Arquitectura de Zabbix: las piezas

Pieza	Qué hace
Zabbix Server + Base de datos	Procesa los datos y los almacena
Frontend web	Dashboards y configuración
Agent / SNMP	Recogen los datos en cada equipo

```
[Equipos con Agent / SNMP] ──datos──► [Zabbix Server + BD] ◄── [Frontend web] ◄── Tú
```

Dos formas de obtener datos:

- **Zabbix Agent:** un pequeño programa que se instala en el equipo (servidor, PC) y le cuenta al servidor cómo va (CPU, memoria, disco...).
- **SNMP:** un protocolo que **ya traen** muchos equipos de red (switches, routers, impresoras, SAI). Aquí **no hay que instalar nada**: Zabbix pregunta y el equipo contesta.

Analogía: el servidor Zabbix es el jefe de planta, la base de datos es su libreta, el frontend es la pantalla de la sala de control, y los agentes/SNMP son los vigilantes que van informando de cada rincón.

6.3 El corazón de Zabbix: Item → Trigger → Problema

Casi todo en Zabbix gira alrededor de **tres conceptos**:

```
📏 Item  ──►  ⚡ Trigger  ──►  🚨 Problema
(mide)        (evalúa)         (genera alerta)
```

Concepto	Qué es	Ejemplo
Item	**Mide un dato**	"Uso de CPU", "estado del puerto 3"

Concepto	Qué es	Ejemplo
Trigger	**Evalúa una condición** sobre los datos	"Si la CPU supera el 90% durante 5 minutos"
Problema	Lo que aparece cuando se cumple el trigger y **genera la alerta**	"CPU alta en el servidor X"

La analogía del termómetro:

- **Item:** el termómetro que mide la temperatura (38,5 °C).
- **Trigger:** la regla *"si pasa de 38 °C, es fiebre"*.
- **Problema:** *"¡Tienes fiebre!"* (y alguien te manda a casa a descansar).

Y un detalle importante: se pueden poner **condiciones inteligentes**, como "durante 5 minutos seguidos", para que un pico de un segundo no os despierte a las 3 de la mañana. Una alerta que salta por todo es una alerta que **nadie mira**.

6.4 Monitorizar un switch por SNMP

Con **SNMP**, el switch responde a preguntas sobre sí mismo. Cada dato tiene un identificador llamado **OID**.

OID	Qué mide
`ifOperStatus`	Estado de la interfaz (1 = up, 2 = down)
`ifInOctets`	Bytes recibidos
`ifOutOctets`	Bytes enviados

Cómo se monta (visión general):

1. Activar SNMP en el switch (con una *community* o, mejor, SNMPv3 con usuario y clave).
2. Dar de alta el switch como **host** en Zabbix.
3. Asociarle una **plantilla** de red SNMP (Zabbix incluye plantillas para muchos fabricantes).
4. Zabbix **descubre automáticamente** las interfaces y empieza a recoger datos.

Ejemplo de qué se puede vigilar:

- ¿Está **caída** una interfaz importante? (`ifOperStatus`)

- ¿Está el enlace **saturado**? (tráfico entrante/saliente frente a la capacidad)
- ¿Hay **muchos errores** o descartes en un puerto? (suele indicar un cable en mal estado)

Consejo de seguridad: nada de dejar la *community* por defecto (`public`). Es como dejar la llave debajo del felpudo con un cartel que dice "la llave está aquí".

> 🤔 **Para lanzar a clase:** ¿qué métrica os interesaría vigilar en el enlace trunk que configuramos en switching?

6.5 Dashboards y alertas

Dashboard: la pantalla donde ves el estado de la red **de un vistazo**.

Dashboard	Acción
Gráficas de tráfico	Trigger → email / chat
Problemas activos	Aviso **también al resolverse**
Mapa de la red	

- **Gráficas de tráfico:** para ver tendencias y picos.
- **Problemas activos:** la lista de lo que está mal *ahora mismo*.
- **Mapa de la red:** el esquema visual con colores (verde = bien, rojo = mal).

Alertas: cuando salta un trigger, Zabbix puede **avisar** por correo, chat y otros medios. Y lo bueno: **avisa también cuando el problema se resuelve**, así sabes que ya pasó.

Buenas prácticas de alertas:

1. **Menos es más.** Si todo es urgente, nada es urgente.
2. Avisa a **quien pueda hacer algo**, no a todo el mundo.
3. Define **niveles de severidad** (información, aviso, crítico).
4. Revisa las alertas que **nadie atiende**: o son inútiles o están mal puestas.

> 😄 **Historia de pasillo:** un administrador ficticio puso alertas para absolutamente todo. A los tres días recibía 400 correos diarios y los había filtrado a una carpeta que no abría nunca. Cuando llegó la alerta *de verdad*, estaba en esa carpeta. Se llama **fatiga de alertas**, y es el equivalente informático a "que viene el lobo".

6.6 Cierre del curso: detecta antes que el usuario

Todo lo visto en el curso se junta en una cadena:

```
🔌 Enlace caído → 📡 SNMP → ⚡ Trigger → 📊 Alerta
```

1. Un cable se rompe o se desconecta (**capa 1**, la de siempre).
2. El switch lo sabe y lo refleja en `ifOperStatus` (**SNMP**).
3. Zabbix lo lee y el **trigger** detecta el cambio a "down".
4. Llega la **alerta** al técnico, que actúa **antes de que llame nadie**.

Y así se cierra el círculo: capas, IP, switching, VLAN, routing... y alguien vigilando que todo eso siga funcionando.

🏆 Proyecto final

Montar una mini-red completa:

- **3 VLAN** (con su direccionamiento, usando lo aprendido de subnetting).
- **Trunk** entre switches.
- **Routing** entre VLAN (router-on-a-stick).
- **Zabbix** vigilando todo.

> 🏁 **Reto final:** ¿qué 3 fallos provocaríais para poner a prueba a un compañero?
> *(Ideas para empezar: desenchufar un cable en el peor momento, quitar una VLAN del trunk, o "olvidar" una ruta de vuelta. Se aplauden la creatividad y la maldad moderada.)*

Mini-glosario del módulo:

- **Host:** equipo monitorizado.
- **Item:** dato que se mide.
- **Trigger:** condición que dispara el aviso.
- **Problema:** alerta generada.
- **SNMP / OID:** protocolo y "dirección" de cada dato en el equipo.
- **Dashboard:** panel de control visual.

Cierre: las 7 reglas del buen técnico de redes

1. **Casi siempre es la capa 1.** Mira el cable.
2. **Casi siempre es el DNS.** Si no es el cable, mira el DNS.
3. **Documenta lo que haces.** El técnico del futuro (que serás tú) te lo agradecerá.
4. **No pruebes en producción** lo que puedas probar antes en un laboratorio.
5. **Cambia una cosa cada vez.** Si cambias cinco y funciona, no sabrás cuál arregló el problema.
6. **Haz copia de seguridad antes de tocar nada.**
7. **Monitoriza.** Es mejor enterarse por una alerta que por una llamada.

> *Y recordad: no existe el "usuario tonto". Existe la red mal explicada.* 😉